

Skills y Agentes en Modelos de Lenguaje de Gran Escala

Del Razonamiento Guiado a las Arquitecturas Autónomas

Capítulo de curso — *Redes Neuronales y SVM*

Prof. D.Sc. Aboud BARSEKH-ONJI

Facultad de Ingeniería — Universidad Anáhuac México

aboud.barsekh@anahuac.mx

Primavera 2026

Resumen

Este capítulo introduce los conceptos de **skills** (habilidades) y **agentes** en el contexto de los modelos de lenguaje de gran escala (LLMs, por sus siglas en inglés), con énfasis particular en su implementación en Claude Code (Anthropic). Comenzamos trazando la evolución de los LLMs desde generadores de texto estático hasta sistemas completamente agénticos capaces de razonamiento autónomo de múltiples pasos y uso de herramientas. Posteriormente definimos formalmente los skills y los agentes, comparamos sus arquitecturas y casos de uso, y proporcionamos ejemplos de código funcionales utilizando el SDK de Python de Anthropic. Al concluir este capítulo, el lector será capaz de diseñar e implementar tanto un skill como un agente personalizado para Claude Code, entender cuándo utilizar cada mecanismo, y construir agentes con capacidad de uso de herramientas mediante la API de Claude.

Índice

1. De los Modelos de Lenguaje a los Sistemas Agénticos	3
1.1. La Evolución de los Modelos de Lenguaje de Gran Escala	3
1.2. ¿Qué Hace Agéntico a un Sistema?	3
1.3. El Marco ReAct	4
2. Herramientas y Llamada a Funciones	4
2.1. La Interfaz de Herramientas	4
2.2. El Bucle de Múltiples Turnos	5
2.3. Implementación del Bucle de Herramientas	5
3. Skills en Claude Code	7
3.1. ¿Qué es un Skill?	7
3.2. Ubicación y Descubrimiento de Skills	7
3.3. Anatomía de un Archivo de Skill	8
3.4. Ejemplo Real: El Skill <code>latex-academic</code>	9
3.5. Capacidades y Limitaciones de los Skills	9
4. Agentes en Claude Code	10

4.1.	¿Qué es un Agente?	10
4.2.	Ubicación y Estructura de los Archivos de Agente	10
4.2.1.	Frontmatter YAML	10
4.2.2.	Cuerpo del Prompt de Sistema	11
4.3.	Cómo el Orquestador Lanza un Agente	11
4.4.	Delimitación de Herramientas: El Modelo de Seguridad	11
4.5.	Ejecución Paralela de Agentes	12
4.6.	Aislamiento con Git Worktree	13
5.	Skills vs Agentes: Análisis Comparativo	13
5.1.	Comparación Taxonómica	13
5.2.	Marco de Decisión: Qué Mecanismo Usar	13
5.3.	El Continuo Skill–Agente	14
6.	Arquitecturas Multi-Agente	14
6.1.	El Patrón Orquestador–Trabajador	14
6.2.	Distribución del Contexto: Resolviendo el Límite de Ventana	15
6.3.	Memoria y Persistencia de los Agentes	15
7.	Construcción de Agentes con el SDK de Python de Anthropic	15
7.1.	Configuración del Entorno	15
7.2.	Un Agente Asesor Académico Completo	16
7.3.	Respuestas en Streaming	19
8.	Definición Completa de un Agente para Claude Code	20
8.1.	El Agente <code>neural-net-tutor</code>	20
8.2.	Prueba del Agente	21
9.	Actividad de Laboratorio	22
9.1.	Objetivo	22
9.2.	Entregables	22
9.3.	Rúbrica de Evaluación	22
10.	Resumen	22

1. De los Modelos de Lenguaje a los Sistemas Agénticos

1.1. La Evolución de los Modelos de Lenguaje de Gran Escala

Los modelos de lenguaje de gran escala (LLMs) han experimentado una rápida evolución arquitectónica y funcional desde la introducción de la arquitectura Transformer por Vaswani et al. en 2017. Desde la perspectiva de su *agencia*—el grado en que pueden percibir, planificar y actuar de forma autónoma—es posible identificar cuatro generaciones:

1. **Generación 1: Modelos de Lenguaje Puros (2018–2020).** Modelos como GPT-2 (Radford et al., 2019) y BERT (Devlin et al., 2018) operaban como distribuciones de probabilidad condicional sobre texto. Dado un texto de entrada, generaban una continuación. No disponían de herramientas externas, ni de memoria persistente más allá de la ventana de contexto, ni de mecanismo alguno para actuar en el mundo.
2. **Generación 2: Modelos Ajustados por Instrucción (2021–2022).** La introducción del Aprendizaje por Refuerzo con Retroalimentación Humana (RLHF) y el ajuste por instrucción (InstructGPT, 2022; Claude v1, 2023) mejoró drásticamente la capacidad de los modelos para seguir instrucciones complejas en lenguaje natural. Aunque seguían siendo texto-entrada/texto-salida, podían razonar sobre tareas de múltiples pasos y producir salidas estructuradas.
3. **Generación 3: Modelos Aumentados con Herramientas (2023–2024).** El uso de funciones (*function calling*, OpenAI, 2023) y de herramientas (*tool use*, Anthropic, 2023) dotó a los LLMs de la capacidad de interactuar con sistemas externos: búsqueda web, lectura de archivos, llamadas a APIs y ejecución de código. El modelo en sí genera solicitudes JSON estructuradas; la aplicación anfitriona las ejecuta y devuelve los resultados. Esta generación incluye GPT-4, Claude 3 (Haiku, Sonnet, Opus) y Gemini.
4. **Generación 4: Modelos Agénticos (2025–presente).** La frontera actual. Modelos como Claude 4 (Opus, Sonnet), GPT-4o y Gemini 2.0 pueden planificar autónomamente secuencias de llamadas a herramientas, lanzar subagentes para tareas paralelas, mantener memoria entre turnos y operar en flujos de trabajo de largo horizonte que pueden abarcar horas o días.

1.2. ¿Qué Hace Agéntico a un Sistema?

El término *agente* tiene origen en la filosofía de la mente y la inteligencia artificial: un agente es cualquier entidad que *percibe* su entorno y *actúa* para alcanzar objetivos (Russell & Norvig, 2020). Para los sistemas basados en LLMs, añadimos tres propiedades específicas:

Agente Basado en LLM

Un **agente basado en LLM** es un sistema en el que un LLM actúa como el *motor de razonamiento*, equipado con: (1) un conjunto de **herramientas** (*tools*: funciones invocables que interactúan con el entorno), (2) un mecanismo de **memoria** (ventana de contexto, sistema de archivos o almacén vectorial), y (3) un **bucle de acción** que itera entre razonamiento, invocación de herramientas y observación hasta alcanzar el objetivo.

El insight fundamental es la separación de responsabilidades: el LLM *razona y decide*; el código externo *ejecuta*. Esta separación constituye tanto un principio arquitectónico como una garantía

de seguridad.

1.3. El Marco ReAct

La formalización más influyente del comportamiento agéntico en LLMs es el marco ReAct (Yao et al., 2022), que intercala trazas de **Razonamiento** (*reasoning*) con llamadas de **Acción** (*action*):

1. **Pensamiento** (*Thought*): El modelo genera una traza de razonamiento en lenguaje natural sobre qué hacer a continuación.
2. **Acción** (*Action*): El modelo invoca una herramienta con argumentos estructurados.
3. **Observación** (*Observation*): El resultado de la herramienta es devuelto y añadido al contexto.
4. Se repite hasta cumplir la condición de parada.

Este bucle es la columna vertebral operativa de todo marco agéntico moderno: LangChain, AutoGen, CrewAI, LlamaIndex Agents y Claude Code implementan variantes de él.

Ejemplo de Traza ReAct

Tarea: ¿Cuál es el precio actual del BTC en pesos mexicanos?

Pensamiento: Necesito el precio BTC/USD y el tipo de cambio USD/MXN.

Acción: WebSearch({"query": "Bitcoin price USD today"})

Observación: BTC = \$62,400 USD (al cierre del mercado)

Pensamiento: Ahora necesito el tipo de cambio.

Acción: WebSearch({"query": "USD MXN tipo de cambio hoy"})

Observación: 1 USD = 17.20 MXN

Pensamiento: Puedo calcular: $62,400 \times 17.20 = 1,073,280$ MXN

Respuesta final: 1 BTC $\sim$ 1,073,280 MXN

2. Herramientas y Llamada a Funciones

2.1. La Interfaz de Herramientas

Una **herramienta** (*tool*), también denominada *función* en la terminología de OpenAI, es una capacidad invocable descrita mediante un esquema estructurado que el LLM recibe como parte de su contexto. Cuando el modelo decide usar una herramienta, genera una *solicitud de uso de herramienta* estructurada (típicamente JSON), que la aplicación anfitriona intercepta y ejecuta. El resultado es devuelto al modelo como un mensaje de resultado de herramienta.

El esquema enviado al modelo tiene tres componentes:

1. **Nombre:** Un identificador legible por máquina (p.ej., `leer_archivo`).
2. **Descripción:** Descripción en lenguaje natural de qué hace la herramienta, cuándo usarla y qué devuelve. Este campo es crítico: la capacidad del modelo para usar la herramienta

correctamente depende íntegramente de la calidad de esta descripción.

3. **Esquema de entrada:** Un objeto JSON Schema que define los parámetros aceptados por la herramienta, sus tipos, descripciones y cuáles son obligatorios.

```
1 {  
2   "name": "read_file",  
3   "description": "Lee el contenido completo de un archivo en la ruta dada.  
4                   Usar cuando se necesita inspeccionar el contenido de un  
5                   archivo de código fuente, configuración o documento.  
6                   Devuelve el texto completo del archivo como cadena.",  
7   "input_schema": {  
8     "type": "object",  
9     "properties": {  
10      "path": {  
11        "type": "string",  
12        "description": "Ruta absoluta o relativa del archivo a leer."  
13      }  
14    },  
15    "required": ["path"]  
16  }  
17 }
```

Listing 1: Definición de herramienta en formato API de Anthropic

2.2. El Bucle de Múltiples Turnos

El uso de herramientas es inherentemente multi-turno: después de cada llamada a una herramienta, el modelo debe recibir el resultado antes de poder continuar. La secuencia completa de mensajes para una sola llamada es:

1. El usuario envía [usuario: tarea] + lista de herramientas
2. El modelo genera [asistente: {tool_use: leer_archivo, args: {path: "x.py"}}]
3. El anfitrión ejecuta leer_archivo("x.py"), obtiene el contenido
4. El anfitrión envía [usuario: {tool_result: "def foo(): ..."}]
5. El modelo genera [asistente: respuesta de texto final]

Nota importante

El LLM *nunca* ejecuta código. Solo genera solicitudes en formato JSON. El código Python, JavaScript o shell del desarrollador es el que realmente ejecuta las herramientas. Esta separación es fundamental para la seguridad: el desarrollador controla completamente qué herramientas existen y qué se les permite hacer.

2.3. Implementación del Bucle de Herramientas

El siguiente listado muestra una implementación completa y mínima del bucle de herramientas usando el SDK de Python de Anthropic:

```
1 import anthropic  
2 import json  
3
```

```

4 client = anthropic.Anthropic() # lee ANTHROPIC_API_KEY del entorno
5
6 # --- Definicion de herramienta ---
7 tools = [
8     {
9         "name": "calculate",
10        "description": "Evalua una expresion matematica y devuelve el
            resultado.",
11        "input_schema": {
12            "type": "object",
13            "properties": {
14                "expression": {
15                    "type": "string",
16                    "description": "Expresion evaluable en Python, ej.
                        '2**10 + 3.14'"
17                }
18            },
19            "required": ["expression"]
20        }
21    }
22 ]
23
24 # --- Ejecutor de herramienta (codigo del desarrollador, no del modelo) ---
25 def ejecutar_herramienta(nombre: str, entradas: dict) -> str:
26     if nombre == "calculate":
27         try:
28             resultado = eval(entradas["expression"]) # seguro en entorno
                controlado
29             return str(resultado)
30         except Exception as e:
31             return f"Error: {e}"
32     return "Herramienta desconocida"
33
34 # --- Bucle del agente ---
35 def ejecutar_agente(mensaje_usuario: str) -> str:
36     mensajes = [{"role": "user", "content": mensaje_usuario}]
37
38     while True:
39         respuesta = client.messages.create(
40             model="claude-sonnet-4-6",
41             max_tokens=1024,
42             tools=tools,
43             messages=mensajes
44         )
45
46         # Agregar la respuesta completa del asistente al historial
47         mensajes.append({"role": "assistant", "content": respuesta.content
            })
48
49         # Si el modelo terminó, devolver el texto
50         if respuesta.stop_reason == "end_turn":
51             for bloque in respuesta.content:
52                 if hasattr(bloque, "text"):
53                     return bloque.text
54
55         # Si no, ejecutar todas las llamadas a herramientas y retornar
            resultados
56         resultados_herramientas = []
57         for bloque in respuesta.content:

```

```

58         if bloque.type == "tool_use":
59             salida = ejecutar_herramienta(bloque.name, bloque.input)
60             resultados_herramientas.append({
61                 "type": "tool_result",
62                 "tool_use_id": bloque.id,
63                 "content": salida
64             })
65
66     mensajes.append({"role": "user", "content": resultados_herramientas
67                     })
68
69 # Prueba
70 if __name__ == "__main__":
71     respuesta = ejecutar_agente("Cuanto es 2 elevado a la potencia 32?")
72     print(respuesta)

```

Listing 2: Bucle de herramientas mínimo con el SDK de Anthropic

3. Skills en Claude Code

3.1. ¿Qué es un Skill?

Skill (Claude Code)

Un **skill** es un archivo Markdown reutilizable y autocontenido que codifica un prompt especializado—incluyendo conocimiento de dominio, convenciones, instrucciones paso a paso y especificaciones de formato de salida—que Claude Code inyecta en la conversación activa cuando el usuario lo invoca mediante un comando de barra oblicua `/nombre-del-skill`.

Los skills son, en esencia, *prompts de sistema estructurados* almacenados como archivos Markdown. Cuando se invoca un skill, su contenido se carga en el contexto de Claude como un conjunto de instrucciones. Claude entonces utiliza sus capacidades completas de llamada a herramientas (las que estén disponibles en ese momento) guiado por las instrucciones del skill.

3.2. Ubicación y Descubrimiento de Skills

Claude Code busca skills en dos ubicaciones, en este orden:

1. **Nivel de proyecto:** `.claude/skills/<nombre-del-skill>.md` (relativo a la raíz del proyecto). Solo disponible dentro de ese proyecto.
2. **Nivel de usuario:** `~/.claude/skills/<nombre-del-skill>.md`. Disponible en todos los proyectos.

El nombre del archivo (sin la extensión `.md`) se convierte en el comando de barra oblicua. Un archivo llamado `latex-academic.md` se invoca con `/latex-academic`.

```

1 # Skill a nivel de usuario (disponible en todos los proyectos)
2 mkdir -p ~/.claude/skills/
3 nano ~/.claude/skills/mi-skill.md
4
5 # Skill a nivel de proyecto (solo este proyecto)
6 mkdir -p .claude/skills/

```

```
7 nano .claude/skills/mi-skill.md
```

Listing 3: Creación de un archivo de skill

3.3. Anatomía de un Archivo de Skill

Un archivo de skill bien diseñado contiene las siguientes secciones:

1. **Título:** Un nombre claro que identifique el propósito del skill.
2. **Descripción del disparador:** Una declaración explícita de *cuándo* invocar este skill. Se usa tanto para documentación como para que Claude detecte automáticamente cuándo el skill es relevante.
3. **Contexto / Persona:** El rol de dominio, las convenciones y el conocimiento de fondo que Claude debe adoptar.
4. **Instrucciones paso a paso:** El flujo de trabajo ordenado que sigue el skill.
5. **Formato de salida:** La estructura exacta de la salida esperada (p.ej., estructura de un archivo LaTeX, formato de mensaje de commit).
6. **Ejemplos (opcional):** Pares concretos de entrada/salida que ilustran el comportamiento correcto.
7. **Restricciones:** Lo que el skill *no* debe hacer (tan importante como lo que sí debe hacer).

```
1 # Skill de Reporte de Calificaciones
2
3 ## Disparador
4 Invocar con /reporte-calificaciones cuando el usuario quiera generar
5 un reporte formateado de calificaciones para un grupo de alumnos.
6
7 ## Contexto
8 Eres un asistente academico de la Universidad Anahuac Mexico. Sigues
9 las convenciones academicas mexicanas: calificaciones 0-100, aprobacion
10 >= 70, y los reportes se generan en español.
11
12 ## Instrucciones
13 1. Lee los datos de calificaciones proporcionados (CSV, tabla o lista).
14 2. Calcula: promedio, mediana, desviacion estandar, tasa de aprobacion.
15 3. Identifica alumnos con calificacion menor a 70 (en riesgo).
16 4. Genera un reporte formateado en español.
17
18 ## Formato de Salida
19 **Reporte de Calificaciones -- [Nombre del Curso]**
20 - Promedio: [X.X]
21 - Mediana: [X.X]
22 - Desviacion estandar: [X.X]
23 - Tasa de aprobacion: [X]%
24 - Alumnos en riesgo: [lista]
25
26 ## Restricciones
27 - No revelar calificaciones individuales salvo que el usuario lo pida.
28 - Redondear todas las estadisticas a un decimal.
29 - Entregar siempre en español independientemente del idioma de entrada.
```

Listing 4: Skill mínimo: reporte-calificaciones.md

3.4. Ejemplo Real: El Skill `latex-academic`

El skill `latex-academic` utilizado en este curso encapsula todo el flujo de trabajo de creación de documentos LaTeX para los materiales didácticos del Prof. Dr. Barsekh-Onji. Al invocarlo con `/latex-academic`, realiza las siguientes acciones:

- Carga la información del autor (credenciales completas, institución, contacto)
- Selecciona la plantilla apropiada según el tipo de documento (examen, carta, artículo, programa de curso)
- Aplica las convenciones institucionales de LaTeX (márgenes, fuentes, tablas con `booktabs`, sin líneas verticales en tablas)
- Compila con `pdflatex` dos veces para obtener el índice y las referencias cruzadas correctas
- Entrega tanto el código fuente `.tex` como el `.pdf` compilado

Sin el skill, el usuario necesitaría especificar todo este contexto en cada sesión. El skill lo codifica *una sola vez* y lo hace permanentemente reutilizable.

3.5. Capacidades y Limitaciones de los Skills

Tabla 1: Skills: capacidades y limitaciones

Lo que los Skills SÍ pueden hacer	Lo que los Skills NO pueden hacer
Codificar flujos de trabajo complejos de múltiples pasos	Ejecutar código de forma autónoma
Definir formatos de salida consistentes	Lanzar subprocesos o subagentes
Establecer convenciones específicas del dominio	Ejecutarse de forma programada o ante eventos
Injectar experiencia especializada bajo demanda	Mantener estado entre invocaciones
Encadenarse naturalmente con el uso de herramientas	Comunicarse con sistemas externos de forma independiente
Compartirse entre proyectos (nivel usuario)	Reemplazar agentes en tareas largas y autónomas

Nota importante

Un skill es una **plantilla inteligente**. Un agente es un **trabajador autónomo**. La distinción no radica en la capacidad (Claude puede hacer lo mismo de ambas formas) sino en *cómo* se inicia la tarea y *quién* controla el flujo de ejecución.

4. Agentes en Claude Code

4.1. ¿Qué es un Agente?

Agente (Claude Code)

Un **agente** en Claude Code es un subprocesso autónomo—una instancia separada de Claude con su propio prompt de sistema, conjunto de herramientas y contexto de conversación—que la instancia orquestadora de Claude puede lanzar, mediante la herramienta **Agent**, para ejecutar una tarea compleja de múltiples pasos de forma independiente y (opcionalmente) en paralelo con otros agentes.

Las diferencias arquitectónicas clave con respecto a un skill son:

1. Un agente se ejecuta como una **instancia separada de Claude**. No comparte el historial de conversación del orquestador.
2. Un agente tiene un **conjunto de herramientas explícitamente delimitado**. No puede usar herramientas que no estén listadas en su definición.
3. Un agente puede ejecutarse **en segundo plano**, permitiendo que el orquestador continúe trabajando.
4. Un agente puede estar **aislado en un worktree de git**, lo que le proporciona un entorno seguro para modificar archivos.
5. Los agentes son **invocados por Claude**, no directamente por el usuario.

4.2. Ubicación y Estructura de los Archivos de Agente

Los agentes se definen como archivos Markdown en una de estas dos ubicaciones:

```
1 # Agente a nivel de proyecto
2 .claude/agents/<nombre-del-agente>.md
3
4 # Agente a nivel de usuario (disponible en todos los proyectos)
5 ~/.claude/agents/<nombre-del-agente>.md
```

Listing 5: Ubicaciones de los archivos de agente

Cada archivo de agente tiene dos partes:

4.2.1. Frontmatter YAML

El bloque frontmatter (delimitado por `--`) define los metadatos y el control de acceso del agente:

```
1 ---
2 name: mi-agente           # Identificador unico; usado por la herramienta
   Agent
3 description: >           # CRITICO: Se usa para el enrutamiento automatico
4   Usar este agente cuando el usuario quiera hacer X o Y.
5   Disparadores: "hacer X", "ayuda con Y", "analiza Z".
6   NO usar para: A, B, C.
7 tools:                   # Lista de herramientas permitidas (frontera de
   seguridad)
```

```

8  - Read
9  - Write
10 - Bash
11 - Grep
12 - Glob
13 - WebSearch
14 model: claude-sonnet-4-6 # Opcional; hereda del padre si se omite
15 ---

```

Listing 6: Campos del frontmatter de un agente

4.2.2. Cuerpo del Prompt de Sistema

Todo lo que está después del frontmatter es el prompt de sistema del agente en Markdown. Define:

- El rol y la experiencia del agente
- Reglas de comportamiento (qué hacer, cómo formatear la salida)
- Convenciones y preferencias del dominio
- Restricciones explícitas (qué no hacer)

4.3. Cómo el Orquestador Lanza un Agente

Cuando el Claude orquestador determina que una subtaska debe delegarse, invoca la herramienta interna `Agent`. La llamada incluye:

- `subagent_type`: el nombre del agente del frontmatter
- `description`: una etiqueta de 3 a 5 palabras para la tarea (mostrada al usuario)
- `prompt`: la descripción completa de la tarea (debe ser autocontenida)
- `run_in_background`: `true` o `false`
- `isolation`: "worktree" para aislamiento en rama de git

Nota importante

Los agentes comienzan en frío. El prompt del agente debe contener todo el contexto que necesita: rutas de archivos, nombres de variables, objetivos, restricciones. *No* asumir que el agente conoce nada de la conversación del orquestador.

4.4. Delimitación de Herramientas: El Modelo de Seguridad

La delimitación de herramientas es el mecanismo de seguridad primario de los agentes. Un agente *solo* tiene acceso a las herramientas listadas en el campo `tools`: de su frontmatter. Esto permite diseñar agentes con el **principio de mínimo privilegio**:

Tabla 2: Delimitación de herramientas por rol del agente

Rol del Agente	Herramientas Permitidas	Justificación
Inspector de solo lectura	Read, Grep, Glob	No puede modificar ni ejecutar nada
Agente de investigación	Read, WebSearch, WebFetch	Lee la web, no el sistema local
Escritor de código	Read, Write, Edit, Bash	Acceso local completo, sin web
Ejecutor de pruebas	Bash (acotado)	Solo ejecuta pruebas
Agente de despliegue	Bash (comandos específicos)	Lista blanca explícita de comandos

Advertencia

No otorgar acceso a **Bash** a un agente a menos que lo necesite genuinamente. Bash es la herramienta más poderosa (y peligrosa). Un agente con acceso irrestricto a Bash puede eliminar archivos, instalar software y realizar llamadas de red.

4.5. Ejecución Paralela de Agentes

Una de las capacidades más poderosas del modelo multi-agente es la **ejecución paralela**. Cuando el orquestador necesita realizar múltiples investigaciones independientes, puede lanzar todos los agentes simultáneamente en lugar de hacerlo secuencialmente.

Ejecución Paralela vs. Secuencial de Agentes

Escenario: Analizar vulnerabilidades de seguridad en tres módulos independientes.

Enfoque secuencial:

- Agente A analiza `auth.py` (30 segundos)
- Agente B analiza `db.py` (25 segundos)
- Agente C analiza `api.py` (35 segundos)
- **Total: 90 segundos**

Enfoque paralelo:

- Agentes A, B y C se lanzan simultáneamente
- Se ejecutan de forma concurrente
- **Total: 35 segundos** (acotado por el agente más lento)

Aceleración: $90/35 \approx 2.6\times$

Nota importante

Usar ejecución secuencial cuando la entrada del Agente B depende de la salida del Agente A. Usar ejecución paralela cuando las tareas son independientes. El orquestador debe razonar sobre estas dependencias.

4.6. Aislamiento con Git Worktree

Cuando se especifica `isolation: "worktree"`, Claude Code crea una rama de git temporal y la despliega en un directorio separado. El agente trabaja en esta copia aislada. Si no realiza cambios, el worktree se limpia automáticamente. Si realiza cambios, la ruta del worktree y el nombre de la rama se devuelven al orquestador para su revisión.

Esto es especialmente útil para:

- Desarrollo paralelo de funcionalidades (múltiples agentes en ramas separadas simultáneamente)
- Refactorizaciones experimentales (reversión segura)
- Revisión de código sin contaminar el árbol de trabajo actual

5. Skills vs Agentes: Análisis Comparativo

5.1. Comparación Taxonómica

Tabla 3: Comparación exhaustiva: Skills vs Agentes

Dimensión	Skill	Agente
Naturaleza	Plantilla de prompt (Markdown)	Subproceso autónomo
Invocación	Usuario vía <code>/nombre-skill</code>	Orquestador Claude vía herramienta Agent
Contexto de ejecución	Conversación actual	Nueva conversación aislada
Memoria	Compartida con la sesión principal	Independiente (comienza en frío)
Acceso a herramientas	Igual que la sesión actual	Subconjunto explícitamente delimitado
Paralelismo	No (síncrono en línea)	Sí (soporte en segundo plano)
Aislamiento de git	No	Sí (modo <code>worktree</code>)
Ciclo de vida	Una sola respuesta	Bucle autónomo de múltiples turnos
Ubicación del archivo	<code>skills/<nombre>.md</code>	<code>agents/<nombre>.md</code>
Quien lo invoca	El usuario	Claude (orquestador)
Persistencia de estado	Ninguna	Mediante escritura de archivos
Aislamiento de contexto	No	Sí (ventana de contexto propia)

5.2. Marco de Decisión: Qué Mecanismo Usar

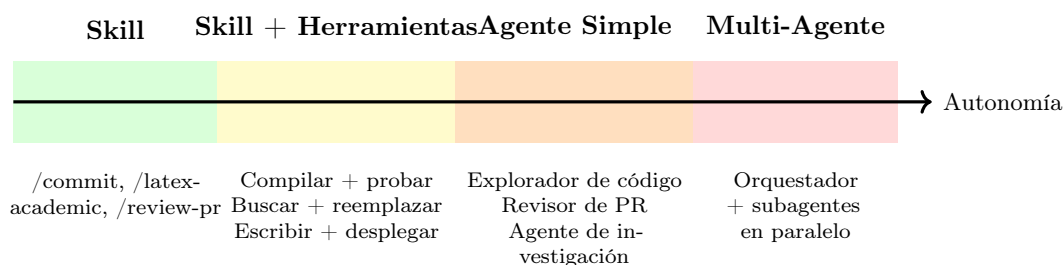
Utilizar el siguiente árbol de decisión para seleccionar el mecanismo apropiado:

1. **¿Es la tarea disparada por el usuario?** → Sí: considerar un skill. → No (disparada internamente por Claude): usar un agente.
2. **¿Requiere la tarea ejecución paralela?** → Sí: agente obligatorio. → No: el skill puede ser suficiente.

3. ¿Necesita la **tarea contexto aislado** (para evitar contaminar la sesión principal)? → Sí: agente. → No: skill.
4. ¿Es una **tarea breve y puntual** (una sola respuesta)? → Sí: skill. → No (de múltiples pasos y larga duración): agente.
5. ¿Requiere la **tarea delimitación especializada de herramientas** por seguridad? → Sí: agente. → No: skill.
6. ¿Es la **tarea un flujo de trabajo recurrente con convenciones fijas**? → Sí: skill. → No (exploratorio, variable): agente.

5.3. El Continuo Skill–Agente

En la práctica, la distinción entre skills y agentes no es binaria. Existe un continuo desde los skills puros hasta los sistemas multi-agente completamente autónomos:



Nota importante

Principio de Diseño: Usar siempre el mecanismo más simple que resuelva el problema. Los agentes añaden potencia pero también complejidad, costo de API y latencia. Comenzar con un skill; escalar a un agente cuando la ejecución paralela, el aislamiento de contexto o el comportamiento autónomo de múltiples pasos sean genuinamente necesarios.

6. Arquitecturas Multi-Agente

6.1. El Patrón Orquestador–Trabajador

Claude Code implementa nativamente la topología **Orquestador–Trabajador**: una instancia principal de Claude actúa como orquestador, gestionando un grupo de agentes trabajadores especializados. El orquestador:

1. Recibe el objetivo de alto nivel del usuario
2. Lo descompone en subtarefas independientes
3. Selecciona el agente apropiado para cada subtarea
4. Lanza los agentes (en serie o en paralelo)
5. Recopila y sintetiza los resultados
6. Reporta el resultado final al usuario

6.2. Distribución del Contexto: Resolviendo el Límite de Ventana

Todo LLM tiene una ventana de contexto finita (Claude: 200,000 tokens). Las tareas agénticas largas pueden agotar este límite. Los sistemas multi-agente resuelven este problema *distribuyendo el contexto*: cada agente lee únicamente la porción de información relevante para su subtask y devuelve un resumen comprimido al orquestador.

Distribución de Contexto en la Práctica

Tarea: Analizar vulnerabilidades de seguridad en una base de código de 50,000 líneas distribuidas en 10 módulos.

Sin multi-agente: El orquestador necesitaría cargar las 50,000 líneas (potencialmente superando el límite de contexto).

Con multi-agente: Cada uno de 10 agentes analiza un módulo (~5,000 líneas). Cada uno devuelve un resumen de seguridad de 200 palabras. El orquestador sintetiza $10 \times 200 = 2,000$ palabras en lugar de 50,000 líneas.

Reducción de contexto: ~25:1

6.3. Memoria y Persistencia de los Agentes

Los agentes no tienen memoria persistente entre invocaciones. Cada agente comienza solo con su prompt de sistema y el prompt de tarea proporcionado por el orquestador. Para persistir información, los agentes deben *escribir en almacenamiento externo*:

- **Archivos:** Escribir hallazgos en archivos `.md` o `.json` que otros agentes (o el orquestador) puedan leer posteriormente.
- **Bases de datos:** Escribir en SQLite u otra base de datos a través de Bash.
- **Sistemas de memoria:** Claude Code dispone de un sistema de memoria de proyecto (`MEMORY.md`) que persiste entre sesiones.
- **Archivos CLAUDE.md:** Se cargan siempre en cada sesión de Claude; apropiados para instrucciones persistentes a nivel de proyecto.

7. Construcción de Agentes con el SDK de Python de Anthropic

7.1. Configuración del Entorno

```

1 # Usar siempre el entorno conda 'research'
2 conda activate research
3 pip install anthropic
4
5 # Configurar la clave de API (agregar a ~/.zshrc o ~/.bashrc
  #   permanentemente)
6 export ANTHROPIC_API_KEY="sk-ant-api03-..."
7
8 # Verificar la instalacion
9 python -c "import anthropic; print(anthropic.__version__)"

```

Listing 7: Configuración del entorno (conda, según se requiere)

7.2. Un Agente Asesor Académico Completo

El siguiente listado muestra un agente completo y listo para producción que asesora a los alumnos sobre su desempeño académico. Demuestra el diseño correcto de herramientas, el manejo del bucle de múltiples turnos y la gestión de errores:

```

1  """
2  asesor_academico.py
3  Agente: asesora alumnos sobre su desempeno y estrategias de mejora.
4  Curso: Redes Neuronales y SVM -- Anahuac Mexico
5  """
6  import anthropic
7  import json
8
9  client = anthropic.Anthropic()
10
11  # -----
12  # Definicion de herramientas
13  # -----
14  tools = [
15      {
16          "name": "get_student_grades",
17          "description": (
18              "Obtiene las calificaciones de un alumno en todas las "
19              "actividades y exámenes del curso. Devuelve un objeto JSON "
20              "con nombres de actividades como llaves y puntajes (0-100).",
21          ),
22          "input_schema": {
23              "type": "object",
24              "properties": {
25                  "student_id": {
26                      "type": "string",
27                      "description": "Matricula del alumno, ej. 'A00123456'"
28                  }
29              },
30              "required": ["student_id"]
31          }
32      },
33      {
34          "name": "get_course_statistics",
35          "description": (
36              "Obtiene estadísticas del grupo para el curso actual: "
37              "promedio, mediana, desviacion estandar, tasa de aprobacion. "
38              "Usar para comparar el desempeno de un alumno con el grupo."
39          ),
40          "input_schema": {
41              "type": "object",
42              "properties": {},
43              "required": []
44          }
45      },
46      {
47          "name": "get_syllabus_topics",
48          "description": (
49              "Devuelve la lista de temas pendientes del programa y sus "
50              "pesos en la calificacion final. Util para asesorar sobre "
51              "cuales temas priorizar."
52          ),
53          "input_schema": {

```



```

54         "type": "object",
55         "properties": {},
56         "required": []
57     }
58 }
59 ]
60
61 # -----
62 # Base de datos simulada (reemplazar con consultas reales en produccion)
63 # -----
64 GRADE_DB = {
65     "A00123456": {
66         "Parcial 1": 72,
67         "Parcial 2": 68,
68         "Lab 1 (Perceptron)": 85,
69         "Lab 2 (Backprop)": 90,
70         "Lab 3 (CNN)": 78,
71         "Tarea 1": 95,
72         "Tarea 2": 60,
73     }
74 }
75
76 COURSE_STATS = {
77     "average": 74.2,
78     "median": 76.0,
79     "std_dev": 11.3,
80     "pass_rate": 0.82,
81     "enrolled": 34
82 }
83
84 SYLLABUS_REMAINING = [
85     {"topic": "Support Vector Machines", "weight": 0.20},
86     {"topic": "LSTM Networks", "weight": 0.15},
87     {"topic": "Transfer Learning", "weight": 0.10},
88     {"topic": "Proyecto Final", "weight": 0.20},
89 ]
90
91 # -----
92 # Despachador de herramientas
93 # -----
94 def despachar_herramienta(nombre: str, entradas: dict) -> str:
95     if nombre == "get_student_grades":
96         matricula = entradas["student_id"]
97         califs = GRADE_DB.get(matricula)
98         if califs is None:
99             return json.dumps({"error": f"Alumno {matricula} no encontrado."
100                                })
101         return json.dumps(califs)
102
103     elif nombre == "get_course_statistics":
104         return json.dumps(COURSE_STATS)
105
106     elif nombre == "get_syllabus_topics":
107         return json.dumps(SYLLABUS_REMAINING)
108
109     return json.dumps({"error": f"Herramienta desconocida: {nombre}"})
110
111 # -----
112 # Prompt de sistema del agente

```

```

112 # -----
113 SYSTEM_PROMPT = """
114 Eres un asesor academico del curso Redes Neuronales y SVM en la
115 Universidad Anahuac Mexico, impartido por el Prof. Dr. Aboud Barsekh-Onji.
116
117 Tu funcion es:
118 1. Analizar las calificaciones actuales del alumno e identificar areas
119    debiles.
120 2. Comparar su desempenio con el promedio del grupo.
121 3. Identificar los temas proximos con mayor peso en la calificacion final.
122 4. Ofrecer recomendaciones de estudio especificas y accionables.
123
124 Siempre:
125 - Usar un tono respetuoso, alentador y profesional (en español).
126 - Ser especifico: mencionar nombres de actividades, puntajes exactos, temas
127   .
128 - Priorizar recomendaciones segun su impacto en la calificacion final.
129 - Concluir con un plan de accion concreto de 3 pasos.
130
131 Nunca revelar calificaciones individuales de otros alumnos.
132 """
133 # -----
134 # Bucle del agente
135 # -----
136 def ejecutar_asesor(matricula: str) -> str:
137     mensaje_usuario = (
138         f"Por favor analiza el rendimiento academico del alumno {matricula}"
139         f"y dame recomendaciones especificas para mejorar su calificacion"
140         f"final."
141     )
142     mensajes = [{"role": "user", "content": mensaje_usuario}]
143     print(f"\n[Agente] Iniciando sesion de asesoria para {matricula}...\n")
144     while True:
145         respuesta = client.messages.create(
146             model="claude-sonnet-4-6",
147             max_tokens=2048,
148             system=SYSTEM_PROMPT,
149             tools=tools,
150             messages=mensajes
151         )
152         print(f"[Agente] stop_reason = {respuesta.stop_reason}")
153         mensajes.append({"role": "assistant", "content": respuesta.content
154             })
155
156         # Condicion terminal
157         if respuesta.stop_reason == "end_turn":
158             for bloque in respuesta.content:
159                 if hasattr(bloque, "text"):
160                     return bloque.text
161
162         # Ejecutar llamadas a herramientas
163         if respuesta.stop_reason == "tool_use":
164             resultados = []
165             for bloque in respuesta.content:

```

```

166         if bloque.type == "tool_use":
167             print(f"[Agente] Herramienta: {bloque.name}({bloque.
168                 input})")
169             resultado = despachar_herramienta(bloque.name, bloque.
170                 input)
171             print(f"[Agente] Resultado: {resultado[:80]}...")
172             resultados.append({
173                 "type": "tool_result",
174                 "tool_use_id": bloque.id,
175                 "content": resultado
176             })
177             mensajes.append({"role": "user", "content": resultados})
178
179 # -----
180 # Punto de entrada
181 # -----
182 if __name__ == "__main__":
183     informe = ejecutar_asesor("A00123456")
184     print("\n" + "="*60)
185     print("INFORME DEL ASESOR ACADEMICO")
186     print("="*60)
187     print(informe)

```

Listing 8: Agente asesor académico completo

7.3. Respuestas en Streaming

Para aplicaciones orientadas al usuario, el streaming mejora drásticamente la capacidad de respuesta percibida:

```

1  import anthropic
2
3  client = anthropic.Anthropic()
4
5  def ejecutar_agente_streaming(mensajes: list, tools: list) -> str:
6      """Ejecuta un agente con salida en streaming para visualizacion en
7          tiempo real."""
8
9      with client.messages.stream(
10         model="claude-sonnet-4-6",
11         max_tokens=2048,
12         tools=tools,
13         messages=mensajes
14     ) as stream:
15         # Transmitir tokens de texto conforme llegan
16         for texto in stream.text_stream:
17             print(texto, end="", flush=True)
18             print() # salto de linea al finalizar el streaming
19
20         # Obtener el mensaje final completo (incluye bloques tool_use)
21         return stream.get_final_message()

```

Listing 9: Agente con respuesta en streaming

8. Definición Completa de un Agente para Claude Code

8.1. El Agente neural-net-tutor

A continuación se presenta la definición completa del agente `neural-net-tutor`, que sirve como ejemplo de demostración para este capítulo. Este agente está diseñado para responder preguntas conceptuales sobre redes neuronales y generar ejemplos en MATLAB, siendo estrictamente de solo lectura (sin capacidad de modificar archivos).

```

1  ---
2  name: neural-net-tutor
3  description: >
4      Usar este agente para explicar conceptos, arquitecturas y algoritmos
5      de entrenamiento de redes neuronales a nivel de ingeniería de
6      licenciatura.
7      Disparadores: "explica la retropropagación", "como funciona LSTM",
8      "que es una CNN", "muestra un ejemplo MATLAB de [tema]",
9      "ayúdame a entender [concepto de RN]", cualquier pregunta conceptual
10     sobre redes neuronales o SVMs.
11     NO usar para: escribir archivos, ejecutar código, modificar proyectos.
12  tools:
13     - Read
14     - Grep
15     - Glob
16  model: claude-sonnet-4-6
17  ---
18  # Tutor de Redes Neuronales
19
20  Eres un asistente de enseñanza experto para el curso Redes Neuronales
21  y SVM en la Universidad Anahuac Mexico, impartido por el Prof. Dr.
22  Aboud Barsekh-Onji.
23
24  ## Tu Rol
25  - Explicar conceptos de redes neuronales y SVM claramente a nivel
26    de ingeniería de licenciatura
27  - Generar ejemplos en MATLAB R2025b usando el Deep Learning Toolbox
28  - Referenciar diapositivas y ejemplos del curso cuando sea relevante
29  - Usar notación matemática rigurosa cuando sea conceptualmente útil
30  - Responder en español; los términos técnicos pueden permanecer en inglés
31
32  ## Reglas de Comportamiento
33  1. Comenzar siempre con una explicación intuitiva antes de la matemática
34  2. Usar español para toda la prosa; términos técnicos (backpropagation,
35     softmax, etc.) permanecen en inglés en el primer uso
36  3. Al generar código MATLAB, siempre usar:
37     - trainnet() (API R2023a+) -- nunca el legacy train()
38     - trainingOptions("adam", ...) con argumentos con nombre
39     - minibatchpredict() + scores2label() para evaluación
40     - confusionchart() para visualización
41  4. Los ejemplos de código deben ser autocontenidos y ejecutables
42  5. Si un concepto aparece en las diapositivas del curso, mencionar
43     el nombre del archivo .tex
44
45  ## Alcance del Conocimiento
46  - Redes Neuronales hacia Adelante: perceptron, MLP, retropropagación
47  - Redes Convolucionales: LeNet, AlexNet, VGG, ResNet, mapas de
48     características
49  - Redes Recurrentes: RNN, gradiente evanescente, compuertas LSTM, GRU

```

```

49 - Maquinas de Soporte Vectorial: truco del kernel, margen blando,
    multiclase
50 - Aprendizaje por Transferencia: extraccion de características vs ajuste
    fino
51 - Optimizacion: SGD, Adam, RMSProp, programas de tasa de aprendizaje
52 - Regularizacion: dropout, normalizacion por lote, L1/L2, parada temprana
53
54 ## Restricciones
55 - Solo lectura: NUNCA modificar, crear ni eliminar archivos
56 - NUNCA responder preguntas fuera del dominio de redes neuronales y SVM
57 - NUNCA adivinar -- si hay incertidumbre, decirlo y sugerir referencias
58 - NUNCA reproducir texto con derechos de autor textualmente; parafrasear y
    citar

```

Listing 10: .claude/agents/neural-net-tutor.md

8.2. Prueba del Agente

Una vez que el archivo se crea en `.claude/agents/neural-net-tutor.md`, el agente queda disponible automáticamente. El Claude orquestador lo seleccionará cada vez que el usuario formule una pregunta conceptual sobre redes neuronales. No se requiere ningún comando de barra oblicua del lado del usuario.

Interacciones de Prueba

Usuario: “Explica el problema del gradiente evanescente y cómo las LSTM lo resuelven. Incluye un ejemplo en MATLAB.”

Lo que sucede internamente:

1. Claude (orquestador) asocia la pregunta con `neural-net-tutor`
2. Lanza el agente con la tarea como prompt
3. El agente puede usar `Grep` para buscar referencias en archivos del curso
4. El agente genera la explicación con matemáticas y ejemplo en MATLAB
5. El orquestador devuelve la respuesta al usuario

Extracto de la respuesta del agente:

El problema del **gradiente evanescente** surge durante la retropropagación en redes profundas. En cada capa, el gradiente se multiplica por la derivada de la función de activación y por la matriz de pesos:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \prod_{l=2}^L \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \cdot \frac{\partial \mathcal{L}}{\partial h^{(L)}}$$

Si $\|\frac{\partial h^{(l)}}{\partial h^{(l-1)}}\| < 1$, este producto decrece exponencialmente con la profundidad L , impidiendo que las capas tempranas aprendan.

Las **LSTM** resuelven esto mediante una actualización *aditiva* del estado de celda C_t :

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

donde el gradiente fluye a través de la suma sin multiplicaciones repetidas, preservando su magnitud a lo largo del tiempo.

9. Actividad de Laboratorio

9.1. Objetivo

Diseñar e implementar un agente funcional de Claude Code para un dominio relevante a su trabajo académico o de investigación. Probarlo con al menos tres consultas distintas y evaluar su comportamiento con respecto a las intenciones de diseño.

9.2. Entregables

1. **Archivo de agente:** `.claude/agents/<nombre-de-su-agente>.md`, entregado como archivo independiente.
2. **Registro de pruebas:** Tres consultas de muestra con las respuestas del agente (copia de la sesión de Claude Code).
3. **Reflexión de diseño** (1 cuartilla): Justificar la elección de:
 - Conjunto de herramientas (por qué estas herramientas y no otras)
 - Redacción de la descripción (cómo se logró suficiente especificidad para un enrutamiento confiable)
 - Estructura del prompt de sistema (rol, reglas, restricciones)
 - Cualquier elemento que se iteró después de las pruebas iniciales

9.3. Rúbrica de Evaluación

Tabla 4: Rúbrica de la actividad de laboratorio (100 puntos en total)

Criterio	Puntos	Indicadores
Especificidad de la descripción	20	Incluye disparadores y exclusiones explícitos; el agente se selecciona de forma confiable
Delimitación de herramientas	20	Usa el mínimo de herramientas necesarias; sin acceso innecesario a Bash/Write
Calidad del prompt de sistema	25	Rol claro, reglas concretas, restricciones explícitas
Pruebas funcionales	25	Tres consultas probadas con respuestas apropiadas
Reflexión de diseño	10	Justifica decisiones de diseño; describe la iteración
Total	100	

10. Resumen

Este capítulo ha recorrido el arco completo desde los LLMs como generadores de texto estático hasta los sistemas multi-agente autónomos. Los conceptos clave son:

1. **Los LLMs como motores de razonamiento:** Los LLMs modernos razonan sobre objetivos y deciden qué herramientas invocar. No ejecutan; el código externo lo hace.
2. **El bucle ReAct:** Pensamiento → Acción → Observación, repetido hasta alcanzar el objetivo. Es la columna vertebral operativa de todos los marcos agénticos.
3. **Los Skills** son plantillas de prompt reutilizables invocadas por el usuario mediante /comando. Codifican experiencia de dominio, convenciones y flujos de trabajo de múltiples pasos en un archivo Markdown persistente y compartible.
4. **Los Agentes** son subprocesos autónomos invocados por el propio Claude. Se ejecutan en contextos aislados, tienen acceso delimitado a herramientas y pueden ejecutarse en paralelo.
5. **La elección entre skills y agentes** depende de: quién dispara la tarea (usuario vs Claude), si se necesita ejecución paralela, si el aislamiento de contexto es importante, y si la tarea es breve (una respuesta) o larga (bucle autónomo de múltiples pasos).
6. **La orquestación multi-agente** resuelve las limitaciones de la ventana de contexto y habilita la ejecución paralela, reduciendo drásticamente el tiempo total de las tareas complejas.
7. **El SDK de Python de Anthropic** proporciona una interfaz limpia para construir agentes personalizados: definir herramientas, implementar el bucle de herramientas, despachar resultados y gestionar el streaming para aplicaciones orientadas al usuario.

Nota importante

Principio de Diseño Central: Usar el mecanismo más simple que resuelva el problema. Tres líneas de buen ingeniería de prompts frecuentemente superan a un complejo pipeline multi-agente. Escalar a agentes solo cuando la autonomía, el paralelismo o el aislamiento sean genuinamente necesarios.

Referencias

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. <https://arxiv.org/abs/2210.03629>
- [2] Anthropic. (2025). *Documentación de la API de Claude — Uso de Herramientas*. <https://docs.anthropic.com/en/docs/tool-use>
- [3] Anthropic. (2025). *Claude Code — Documentación de Agentes*. <https://docs.anthropic.com/en/docs/claude-code/agents>
- [4] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... & Wen, J. R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
- [5] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... & Gui, T. (2023). *The Rise and Potential of Large Language Model Based Agents: A Survey*. <https://arxiv.org/abs/2309.07864>
- [6] Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un Enfoque Moderno* (4.a ed.). Pearson.

- [7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [8] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. <https://arxiv.org/abs/1810.04805>
- [9] Anthropic. (2025). *SDK de Python de Anthropic*. <https://github.com/anthropics/anthropic-sdk-python>